

Stredná priemyselná škola informačných technológií Ignáca Gessaya,
Stredná odborná škola lesnícka
a Gymnázium
Medvedzie 133/1, 027 44 Tvrdošín

Online testovací modul pre maturantov

Stredná priemyselná škola informačných technológií Ignáca Gessaya,
Stredná odborná škola lesnícka
a Gymnázium
Medvedzie 133/1, 027 44 Tvrdošín

Online testovací modul pre maturantov

Študijný odbor: 3918 M technické lýceum

Trieda: IV.C

Konzultant práce: Mgr. Martin Ovšák



ŽILINSKÝ
samosprávny kraj
zriaďovateľ

Stredná priemyselná škola informačných technológií Ignáca Gessaya,
Stredná odborná škola lesnícka
a Gymnázium
Medvedzie 133/1, 027 44 Tvrdošín



ZADANIE VLASTNÉHO PROJEKTU

pre

PČOZ MS forma b)

v školskom roku 2025/2026

Názov vlastného projektu: Online testovací modul pre maturantov

Meno riešiteľa: Matúš Čiernik

Trieda: 4.C

Študijný odbor: 3918 M Technické lýceum

Meno konzultanta: Mgr. Martin Ovšák

Cieľ vlastného projektu: Cieľom projektu je vytvoriť online nástroj, ktorý pomôže maturantom efektívne sa pripraviť na maturitnú skúšku. Systém umožní riešiť testy, zobrazovať správne odpovede, ukladať štatistiky úspešnosti.

Stručná anotácia:

Prístupnosť a responzivita – z PC aj mobilu

Delenie otázok podľa predmetov

História úspešnosti riešenia testov

Zobrazenie správnych odpovedí a percentá úspešnosti po dokončení testu

Spôsob využitia: Projekt môžu využívať maturanti, poprípade aj ostatní žiaci pri príprave na maturitné skúšky alebo aj písomky.

Finančné, technologické a priestorové nároky na realizáciu projektu: 0€

Tvrdošín, 20. september 2025

PaedDr. Peter Spišský
riaditeľ poverený vedením

Čestné vyhlásenie a pod'akovanie

Vyhlasujem, že som zadanú prácu vypracoval samostatne, pod odborným vedením vedúceho práce Mgr. Martina Ovšáka a používal som len literatúru uvedenú v práci.

Súhlasím s tým, že práca bude uložená v školskej knižnici a môže byť zapožičaná záujemcom o jej preštudovanie.

Tvrdošín, 9. 2. 2026

podpis

Abstrakt

ČIERNIK, Matúš: Online testovací modul pre maturantov. Stredná priemyselná škola informačných technológií Ignáca Gessaya, Tvrdošín: 2026. Počet strán: 18, počet znakov: 24 194.

Táto práca sa zaoberá vytvorením online testovacieho modulu určeného pre maturantov stredných škôl. Hlavným cieľom projektu je pomôcť študentom pripraviť sa na maturitnú skúšku jednoduchou a prehľadnou formou. Systém umožňuje riešiť testy z rôznych predmetov, zobrazíť vyhodnotenie testov priamo po dokončení a sledovať úspešnosť používateľa. Aplikácia je prístupná z počítača aj mobilu a je jednoduchá a intuitívna. Projekt reflektuje rastúcu potrebu využívania moderných technológií vo vzdelávaní.

Kľúčové slová: maturitná práca, online testovací modul, webová aplikácia

Abstract

ČIERNIK, Matúš: Online Test Module for High School Graduates. Ignác Gessay Secondary School of Information Technology, Tvrdošín: 2026. Number of pages: 18, number of characters: 24 194.

This thesis focuses on the development of an online testing module intended for secondary school graduation students. The main goal of the project is to help students prepare for the final graduation exam in a simple and clear way. The system allows students to complete tests from various subjects, view test evaluations immediately after completion, and track their performance. The application is accessible on both desktop and mobile devices and is designed to be simple and intuitive. The project reflects the growing need for the use of modern technologies in education.

Keywords: graduation thesis, online test module, web application

Obsah

Úvod	13
1 Zadanie a cieľ projektu	14
1.1 Zadanie maturitného projektu	14
1.2 Cieľ projektu	14
1.3 Požiadavky na projekt.....	15
2 Použité technológie	16
2.1 Použité programovacie jazyky	16
2.1.1 Programovací jazyk C#.....	16
2.1.2 Programovací jazyk TypeScript.....	16
2.2 Použité knižnice a frameworky.....	17
2.2.1 Backend.....	17
2.2.2 Frontend	18
2.3 Databáza a ukladanie údajov.....	20
2.4 Použité vývojové nástroje.....	20
3 Návrh systému.....	21
3.1 Celkový návrh systému	21
3.2 Architektúra aplikácie	21
3.2.1 Backendová architektúra.....	21
3.2.2 Frontendová architektúra	22
3.2.3 Databázová technológia	23
3.3 Návrh používateľských rolí a oprávnení	23
4 Implementácia systému.....	24
4.1 Implementácia backendu	24
4.1.1 Štruktúra backend projektu	24
4.1.2 Implementácia controllerov	25
4.1.3 Implementácia servisov.....	25
4.1.4 Autentifikácia a autorizácia používateľov.....	25
4.2 Implementácia frontendu.....	26
4.2.1 Štruktúra frontend projektu	26
4.2.2 Routing.....	27
4.2.3 UI a UX.....	27
4.2.4 Formuláre a validácia.....	28

4.2.5	Správa dát.....	28
4.2.6	Autentifikácia a autorizácia.....	28
4.2.7	Typová bezpečnosť	28
5	Deploy aplikácie	29
	Záver	30
	Príloha A.....	A

Zoznam skratiek

Skratka	Anglický význam	Slovenský význam
API	Application programming interface	Rozhranie pre programovanie aplikácií
UI	User interface	Používateľské rozhranie
ORM	Object-relational mapping	Objektovo relačné mapovanie
UX	User Experience	Používateľská skúsenosť
DTO	Data transfer object	
C#	C sharp	
HTTP	Hypertext transfer protocol	
JWT	Json web token	

Úvod

Maturitná skúška je najdôležitejšou skúškou na strednej škole a jej zvládnutie si vyžaduje dôkladnú prípravu. Mnohí študenti sa pri učení stretávajú s problémom, že nemajú dostatok testov alebo nemajú možnosť overiť si svoje vedomosti rýchlo a prehľadne. Práve preto je vhodné využívať online nástroje, ktoré umožňujú efektívnejšie učenie.

V minulosti sa testovanie realizovalo najmä formou písomných testov alebo pracovných zošitov. Postupne sa však začali používať počítače a internet, čo umožnilo vznik rôznych vzdelávacích webových stránok. Aj keď dnes existuje viacero online platforiem, nie všetky sú zamerané priamo na maturitné otázky alebo poskytujú prehľad o dlhodobej úspešnosti študenta.

Cieľom tejto práce je vytvoriť online testovací modul pre maturantov, ktorý umožní riešenie testov rozdelených podľa predmetov, zobrazenie správnych odpovedí a vyhodnotenie úspešnosti. Ďalším cieľom je vytvoriť jednoduchý a prehľadný systém, ktorý bude dostupný z rôznych zariadení a bude vhodný na samostatnú prípravu na maturitu.

1 ZADANIE A CIEĽ PROJEKTU

Táto kapitola sa zameriava na analýzu zadania maturitného projektu a stanovenie hlavných cieľov vytvárajanej aplikácie. V úvode sú definované požiadavky na systém a očakávané funkcie aplikácie. Cieľom kapitoly je objasniť, aký problém projekt rieši, aké sú jeho ciele a aké požiadavky musí výsledné riešenie spĺňať.

1.1 ZADANIE MATURITNÉHO PROJEKTU

Zadanie maturitného projektu spočíva vo vytvorení webovej aplikácie určenej na tvorbu, správu a vyplňovanie testov. Aplikácia má slúžiť ako jednoduchý testovací systém, ktorý umožní vytváranie otázok, ich zoskupovanie do testov a následné vyhodnocovanie odpovedí používateľov. Systém má byť rozdelený na serverovú a klientsku časť. Serverová časť zabezpečuje spracovanie údajov, komunikáciu s databázou a poskytovanie aplikačného rozhrania (API). Klientska časť zabezpečuje používateľské rozhranie, prostredníctvom ktorého môže používateľ so systémom pracovať. Súčasťou zadania je aj návrh databázovej štruktúry na ukladanie otázok, testov a výsledkov testovania. Projekt má byť realizovaný s využitím moderných technológií a má preukázať schopnosť študenta navrhnuť, implementovať a zdokumentovať funkčný softvérový systém.

1.2 CIEĽ PROJEKTU

Hlavným cieľom maturitného projektu je navrhnuť a implementovať webový testovací systém, ktorý umožní efektívnu tvorbu, správu a vyplňovanie testov prostredníctvom webového rozhrania. Systém má poskytovať jednoduché a prehľadné ovládanie pre používateľa a zároveň zabezpečovať spoľahlivé spracovanie a ukladanie údajov. Cieľom projektu je umožniť administrátorovi vytvárať rôzne typy otázok, zoskupovať ich do testov a spravovať ich obsah. Používateľom má systém umožniť vyplňovanie testov a zobrazenie výsledkov po ich dokončení. Aplikácia má automaticky vyhodnocovať odpovede a ukladať výsledky do databázy. Ďalším cieľom projektu je preukázať schopnosť študenta pracovať s modernými webovými technológiami, navrhovať databázovú štruktúru a vytvárať prepojenie medzi frontendovou a

backendovou časťou aplikácie. Projekt má zároveň rozvíjať schopnosti samostatnej práce, analytického myslenia a riešenia problémov počas vývoja softvéru.

1.3 POŽIADAVKY NA PROJEKT

Požiadavky na systém sú rozdelené na funkčné a nefunkčné. Tieto požiadavky definujú správanie a vlastnosti vytvorenej aplikácie.

Funkčné požiadavky

- systém umožňuje registráciu a prihlásenie používateľov
- systém rozlišuje používateľské roly (napr. administrátor, učiteľ, používateľ)
- učiteľ môže spravovať vyučovacie predmety
- učiteľ môže vytvárať, upravovať a odstraňovať otázky
- učiteľ môže vytvárať testy a priradovať k nim otázky
- používateľ môže vyplniť dostupný test
- systém automaticky vyhodnotí test po jeho odoslaní
- systém uloží výsledky testu do databázy
- používateľ si môže zobrazit' výsledok vyplneného testu
- administrátor môže spravovať používateľov, priradovať im používateľské roly, pomôcť s prístupom k účtu

Nefunkčné požiadavky

- aplikácia musí mať prehľadné a intuitívne používateľské rozhranie (UI)
- systém musí zabezpečovať ochranu používateľských údajov
- aplikácia musí byť dostupná prostredníctvom webového prehliadača
- systém musí byť stabilný a spoľahlivý
- aplikácia musí reagovať na používateľské akcie v primeranom čase

2 POUŽITÉ TECHNOLOGIE

Pri vývoji webového testovacieho systému boli použité moderné technológie, ktoré umožňujú tvorbu spoľahlivej, bezpečnej a rozšíriteľnej aplikácie. Aplikácia je rozdelená na backendovú a frontendovú časť, pričom každá z nich využíva vhodné nástroje a technológie.

2.1 *POUŽITÉ PROGRAMOVACIE JAZYKY*

Na implementáciu serverovej časti aplikácie bol použitý programovací jazyk C#, ktorý je súčasťou platformy .NET. Na tvorbu klientskeho rozhrania bol použitý programovací jazyk TypeScript.

2.1.1 Programovací jazyk C#

Na implementáciu serverovej časti aplikácie bol použitý programovací jazyk C#, ktorý je súčasťou platformy .NET. Tento jazyk je vhodný na vývoj webových aplikácií vďaka svojej výkonnosti, bezpečnosti a dobrej podpore objektovo-orientovaného programovania. Jazyk C# umožňuje jednoduchú tvorbu aplikačných rozhraní (API), spracovanie požiadaviek zo strany klienta a prácu s databázou prostredníctvom moderných knižníc. V projekte bol použitý na implementáciu backendovej logiky, autentifikácie používateľov a komunikácie s databázou.

2.1.2 Programovací jazyk TypeScript

Na tvorbu klientskeho rozhrania aplikácie bol použitý programovací jazyk TypeScript, ktorý rozširuje jazyk JavaScript o statické typovanie. Vďaka tomu je možné odhaliť mnohé chyby už počas vývoja a zvýšiť celkovú kvalitu zdrojového kódu. Použitie TypeScriptu prispieva k lepšej čitateľnosti a udržiavateľnosti kódu, čo je dôležité najmä pri väčších projektoch. V projekte bol TypeScript využitý pri tvorbe frontendovej časti aplikácie a pri komunikácii s backendovým API.

2.2 *POUŽITÉ KNIŽNICE A FRAMEWORKY*

Pri vývoji bolo využitých viacero moderných frameworkov a knižníc, ktoré nám pomohli zjednodušiť vývoj aplikácie a zároveň zabezpečiť jej spoľahlivosť a dobrý používateľský zážitok. Výber technológií bol prispôsobený požiadavkám projektu a snažil som sa použiť nástroje, s ktorými mám skúsenosti a pracuje sa mi s nimi dobre a zároveň také, ktoré sa bežne používajú pri vývoji webových aplikácií.

2.2.1 Backend

Backend predstavuje serverovú časť aplikácie, v ktorej sa nachádza hlavná logika celého systému. V tejto časti aplikácie sa spracovávajú požiadavky od používateľov, vykonáva sa komunikácia s databázou a rieši sa autentifikácia a autorizácia používateľov. Backend zabezpečuje, aby boli všetky údaje správne spracované, uložené a chránené pred neoprávneným prístupom. Zároveň poskytuje aplikačné rozhranie (API), prostredníctvom ktorého komunikuje frontend s backendom. Frontend odosiela požiadavky na server a backend na ne reaguje spracovaním dát a odoslaním odpovedí.

2.2.1.1 *.NET Core*

Na vývoj serverovej časti aplikácie bol použitý framework ASP.NET Core, ktorý je súčasťou platformy .NET. Tento framework umožňuje vytvoriť webové API, cez ktoré frontend komunikuje s backendom. Pomocou neho je možné spracovávať požiadavky od používateľov, riešiť autentifikáciu a pracovať s databázou. Výhodou ASP.NET Core je jeho výkon, bezpečnosť a prehľadná štruktúra projektu.

2.2.1.2 *Entity Framework Core*

Na ukladanie a čítanie dát z databázy je použitý Entity Framework Core. Ide o knižnicu, ktorá umožňuje pracovať s databázou pomocou objektov v jazyku C#, bez potreby písať zložité SQL dotazy. Vďaka tomu bol kód prehľadnejší a jednoduchšie sa udržiaval. Entity Framework Core môžeme využiť napríklad pri ukladaní používateľov, otázok, testov a výsledkov testovania.

2.2.1.3 *JWT*

Autentifikáciu a autorizáciu používateľov som riešil pomocou JWT (Json Web Token). Pomocou autentifikácie a autorizácie som docielil, aby mohol používateľ pracovať s chránenými časťami aplikácie, ale iba v rámci svojich oprávnení. Po úspešnom prihlásení sa používateľovi vygeneruje token, ktorý sa následne posiela spolu s každou požiadavkou na server. Vďaka tomu je možné overiť identitu používateľa a zabezpečiť ochranu jeho údajov.

2.2.2 Frontend

V rámci frontendovej časti aplikácie som použil viacero knižníc a nástrojov, ktoré mi výrazne zjednodušili vývoj používateľského rozhrania. Tieto knižnice mi pomohli najmä pri komunikácii so serverom prostredníctvom API, správe dát v aplikácii a pri štylovaní jednotlivých častí používateľského rozhrania. Vďaka ich využitiu je aplikácia prehľadná, jednotná a jednoduchá na používanie.

2.2.2.1 *React a Next.js*

Na tvorbu používateľského rozhrania som použil knižnicu React spolu s programovacím jazykom TypeScript a frameworkom Next.js. React slúži na vytváranie dynamických webových stránok a umožňuje rozdeliť aplikáciu na menšie, samostatné komponenty, ako sú formuláre, zoznamy alebo jednotlivé obrazovky aplikácie. Tento prístup zjednodušuje úpravy kódu a ďalší vývoj aplikácie. Next.js som použil ako nadstavbu nad Reactom. Umožňuje jednoduché smerovanie (routing) medzi stránkami aplikácie a zlepšuje celkový výkon webovej aplikácie. Vďaka Next.js je štruktúra projektu prehľadnejšia a aplikácia sa rýchlejšie načítava, čo má pozitívny vplyv na používateľský komfort.

2.2.2.2 *Tanstack Query a Axios*

TanStack Query a Axios som využíval na prácu z dátami na frontende. Axios slúži na odosielanie HTTP požiadaviek na backendové API, napríklad pri prihlasovaní

používateľa, načítavanie testov alebo ukladanie výsledkov. Je jednoduchý na používanie a umožňuje prehľadné spracovanie odpovedí zo servera. TanStack Query som použil na správu dát získaných zo servera. Táto knižnica automaticky rieši ukladanie dát do pamäte (caching), ich opätovné načítanie a aktualizáciu. Vďaka tomu má aplikácia vždy aktuálne údaje a práca s dátami je prehľadnejšia.

2.2.2.3 *Tanstack Form a Zod*

Knižnica TanStack Form mi pomohla so správou formulárových vstupných polí a ich stavov. Táto knižnica umožňuje jednoducho spracovať údaje z formulárov, validovať ich a vypísať používateľovi, čo vyplnil zle. Použitie TanStack Form prispelo k prehľadnosti kódu a zjednodušilo validáciu vstupných údajov. Na validáciu jednotlivých vstupných polí (inputov) som využil ďalšiu knižnicu Zod, ktorá umožňuje presne určiť, aké hodnoty môže používateľ zadať. Vďaka kombinácii TanStack Form a Zod je možné odhaliť chyby už pri vyplňaní formulára a upozorniť používateľa. To zabezpečí, že do systému sa dostanú iba správne údaje.

2.2.2.4 *UI a UX Aplikácie*

Na tvorbu používateľského rozhrania som použil komponentovú knižnicu shadcn/ui, ktorá poskytuje hotové a štýlovo jednotné komponenty, ako sú tlačidlá, formuláre, dialógové okná alebo tabuľky. Tieto komponenty mi umožnili rýchlejšie vytvoriť konzistentné používateľské rozhranie bez potreby vytvárať všetky prvky od začiatku. Na doplnenie používateľského rozhrania o ikony slúžila knižnica Lucide React. Ikony zlepšujú prehľadnosť aplikácie a uľahčujú používateľovi orientáciu v jednotlivých častiach systému. Na štýlovanie aplikácie som použil Tailwind CSS, čo je náhrada za css, ktorá umožňuje rýchle a efektívne vytváranie moderného a responzívneho dizajnu. Pomocou tejto knižnice som dokázal jednoducho prispôsobiť vzhľad aplikácie rôznym veľkostiam obrazoviek. Tailwind CSS zároveň zabezpečuje jednotný vzhľad aplikácie a zjednodušuje údržbu štýlov.

2.3 *DATABÁZA A UKLADANIE ÚDAJOV*

Na ukladanie údajov bola použitá relačná databáza PostgreSQL, ktorá poskytuje vysokú spoľahlivosť a výkon pri práci s väčším množstvom dát. Databáza slúži na ukladanie všetkých dát v aplikácii. Komunikácia medzi backendovou aplikáciou a databázou je realizovaná pomocou objektovo-relačného mapovania (ORM), v mojom prípade Entity Framework Core, ktoré je s .NET aplikáciami veľmi často využívané. ORM umožňuje prevod relácií (tabuliek v databáze) na objekty a tým umožňuje jednoduchú a prehľadnú prácu s databázovými tabuľkami priamo v programovacom jazyku.

2.4 *POUŽITÉ VÝVOJOVÉ NÁSTROJE*

Na vývoj backendovej časti aplikácie bolo použité vývojové prostredie Visual Studio, ktoré poskytuje nástroje na jednoduchý vývoj aplikácie. Frontendová časť bola vyvíjaná v prostredí Visual Studio Code. Na správu zdrojového kódu bol použitý Github Desktop, ktorý umožňuje sledovanie zmien a spoluprácu na projekte. Na testovanie aplikácie bol použitý webový prehliadač a nástroje určené na kontrolu správnosti funkčnosti systému.

3 NÁVRH SYSTÉMU

V tejto kapitole sa zaoberám návrhom webového testovacieho systému ešte pred samotnou implementáciou. Cieľom návrhu systému bolo vytvoriť prehľadnú, rozšíriteľnú a bezpečnú aplikáciu, ktorá bude jednoduchá na používanie pre študentov aj administrátorov. V kapitole je popísaná celková architektúra systému, návrh databázy, používateľské roly, komunikácia medzi frontendovou a backendovou časťou a základné bezpečnostné princípy aplikácie.

3.1 CELKOVÝ NÁVRH SYSTÉMU

Navrhovaná aplikácia je webový systém typu klient–server. Systém je rozdelený na tri základné časti – frontend, backend a databázu. Frontend predstavuje používateľské rozhranie aplikácie, ktoré je dostupné prostredníctvom webového prehliadača. Backend zabezpečuje spracovanie logiky aplikácie, overovanie používateľov, prácu s databázou a poskytovanie aplikačného rozhrania (API). Databáza slúži na ukladanie všetkých potrebných údajov, ako sú používatelia, predmety, otázky, testy a výsledky testovania. Komunikácia medzi frontendom a backendom prebieha pomocou HTTP požiadaviek cez REST API. Backend spracuje požiadavku, vykoná potrebnú logiku a odošle odpoveď späť klientovi. Tento spôsob návrhu umožňuje jednoduché rozšírenie systému a oddelenie jednotlivých častí aplikácie.

3.2 ARCHITEKTÚRA APLIKÁCIE

Architektúra aplikácie bola navrhnutá tak, aby bola prehľadná, logicky rozdelená a jednoducho udržiavateľná. Každá časť aplikácie má jasne definovanú zodpovednosť.

3.2.1 Backendová architektúra

Backendová časť aplikácie je navrhnutá na základe vrstvenej architektúry. Tento architektonický prístup rozdeľuje aplikáciu do viacerých vrstiev, pričom každá vrstva má presne určenú úlohu. Základnými vrstvami backendu sú kontroléry (Controllers) a servisná vrstva (Services). Controllery slúžia ako vstupný bod do aplikácie. Prijímajú HTTP požiadavky z frontendovej časti aplikácie a následne odovzdávajú požiadavky do

servisnej vrstvy. V controlleroch sa zároveň vykonáva aj autorizácia používateľov, čo znamená overenie, či je daný používateľ oprávnený vykonávať požadovanú operáciu alebo pristupovať ku konkrétnym častiam systému na základe svojej používateľskej roly. Controllery neobsahujú žiadnu aplikačnú logiku, vďaka čomu zostávajú jednoduché a prehľadné. Servisná vrstva obsahuje hlavnú logiku aplikácie. Táto vrstva obsahuje čistý C# kód a nerieši sa tu komunikácia s používateľským rozhraním ani spracovanie HTTP požiadaviek. V tejto vrstve sa rieši komunikácia z databázou prostredníctvom knižnice Entity Framework Core, ukladanie dát ako aj validácia dát zadaných používateľom. Oddelenie tejto logiky do samostatnej vrstvy umožňuje jej prehľadnosť a jednoduchšie opätovné použitie.

3.2.2 Frontendová architektúra

Frontendová časť aplikácie je navrhnutá pomocou komponentového prístupu, typického pre knižnicu React. Používateľské rozhranie (UI) je rozdelené na menšie, znovupoužiteľné komponenty, ktoré v sebe spájajú štruktúru stránky, štylovanie aj funkcionality. Tento prístup umožňuje prehľadnú organizáciu kódu, jednoduchšiu údržbu a rýchle úpravy jednotlivých častí používateľského rozhrania bez zásahu do celej aplikácie. Z týchto komponentov potom vykladáme jednotlivé stránky, ktoré vďaka tomu majú jednotný dizajn a predvídateľnú funkcionality. Framework Next.js slúži ako nadstavba nad Reactom a zabezpečuje smerovanie medzi jednotlivými stránkami aplikácie. Vďaka zabudovanému routingu je možné jednoducho vytvárať nové stránky a prechádzať medzi nimi. Next.js zároveň prispieva k lepšiemu výkonu aplikácie tým, že optimalizuje načítavanie stránok a zdrojov, čo zlepšuje používateľský komfort. Správa dát na frontende je riešená pomocou knižníc Axios a TanStack Query. Tieto knižnice zabezpečujú efektívne načítavanie dát zo servera, ich ukladanie do pamäte a automatickú aktualizáciu pri zmene údajov. Vďaka tomu má aplikácia vždy aktuálne informácie bez potreby manuálneho obnovovania stránok. Tento prístup znižuje počet zbytočných požiadaviek na server a zlepšuje celkový výkon aplikácie. Frontend architektúra je navrhnutá tak, aby bola prehľadná, ľahko rozšíriteľná a aby poskytovala plynulý a intuitívny používateľský zážitok.

3.2.3 Databázová technológia

Pre ukladanie dát je využitá databáza PostgreSQL. Je to SQL databáza a na základe toho je presne vhodná pre môj projekt. Môj backend s touto databázou komunikuje pomocou ORM Entity Framework Core. Použitie ORM nástroja umožňuje pracovať s databázovými tabuľkami pomocou objektov v jazyku C#, čím eliminujeme potrebu písania vlastných SQL dotazov. Takýto prístup zvyšuje bezpečnosť aplikácie a zjednodušuje údržbu databázovej logiky.

3.3 *NÁVRH POUŽÍVATEĽSKÝCH ROLÍ A OPRÁVNENÍ*

Systém je navrhnutý tak, aby podporoval viacero používateľských rolí. Každá rola má presne definované oprávnenia, vďaka čomu má používateľ prístup len k tým funkciám aplikácie, ku ktorým je oprávnený. V tejto aplikácii sú role Admin, Teacher, User. User môže vypracúvať testy, vidieť svoje štatistiky úspešnosti a pripravovať sa na skúšky aj pomocou náhodne vygenerovaných testov z daného predmetu. Teacher môže navyše spravovať databázu predmetov a otázok, pridávať testy študentom a vidieť celkové štatistiky úspešnosti všetkých študentov. Admin môže spravovať používateľské roly všetkých používateľov a zároveň na žiadosť používateľa zmeniť jeho zabudnuté heslo.

4 IMPLEMENTÁCIA SYSTÉMU

V tejto kapitole je opísaná implementácia tohto systému na základe navrhnutého riešenia. Kapitola sa zameriava na realizáciu backendovej a frontendovej časti aplikácie, ich vzájomnú komunikáciu a spracovanie jednotlivých funkcií systému. Cieľom je ukázať, ako bol návrh systému prevedený do funkčnej aplikácie.

4.1 *IMPLEMENTÁCIA BACKENDU*

4.1.1 Štruktúra backend projektu

Implementáciu som začal vytvorením si štruktúry priečinkov, aby bol projekt prehľadný a dobre organizovaný už od začiatku. Cieľom bolo rozdeliť jednotlivé časti aplikácie podľa ich zodpovednosti a oddeliť aplikačnú logiku od spracovania požiadaviek a práce s databázou.

Projekt je rozdelený do viacerých základných priečinkov, ako sú Controllers, Services, Entities, Data, DTOs, Exceptions a Middleware.

Priečink Controller obsahuje triedy, ktoré prijímajú HTTP požiadavky z frontendu a slúžia ako vstupný bod do aplikácie.

V priečinku Services sa nachádza hlavná aplikačná logika systému, napríklad správa testov, otázok, predmetov a používateľov.

Priečink Entities obsahuje triedy, ktoré reprezentujú databázové tabuľky a slúžia na ukladanie údajov do databázy.

V priečinku Data sa nachádza databázový kontext, ktorý zabezpečuje komunikáciu medzi aplikáciou a databázou pomocou Entity Framework Core.

Priečink DTOs obsahuje triedy, ktoré slúžia na prenos dát medzi backendom a frontendom. Vďaka DTO objektom nie je potrebné posilať celé databázové entity, ale iba tie údaje, ktoré sú pre danú operáciu potrebné.

Priečink Exceptions obsahuje vlastné výnimky, ktoré sa používajú pri chybových stavoch v aplikácii, napríklad pri neplatných vstupných údajoch alebo pri pokuse o prístup k neexistujúcim dátam. Tieto vlastné výnimky sú navrhnuté tak, že pri ich

vyhodení (throw) v sebe nesú aj príslušný HTTP stavový kód a message. Vďaka tomu je v aplikácii jednoduché zistiť, prečo daná chyba nastala.

V priečinku Middleware sa nachádzajú triedy, ktoré zachytávajú a spracovávajú požiadavky ešte predtým, ako sa dostanú do controllerov, alebo odpovede pred ich odoslaním klientovi. Middleware som využil konkrétne na globálne spracovanie chýb a na zabezpečenie jednotných chybových hlások zo servera.

4.1.2 Implementácia controllerov

Controllery sú implementované ako triedy, ktoré dedia z ControllerBase. Každý controller zodpovedá konkrétnej časti aplikácie, napríklad používateľom, testom, predmetom alebo otázkam. Ich úlohou je prijímať HTTP požiadavky z frontendu, overiť oprávnenia používateľa a odovzdať požiadavku servisnej vrstve. V controlleroch sa nachádzajú metódy označené atribútmi ako `HttpGet`, `HttpPost`, `HttpPut` a `HttpDelete`, ktoré určujú typ HTTP metódy. Controllery neobsahujú samotnú aplikačnú logiku, ale slúžia iba ako sprostredkovateľ medzi frontendom a servisnou vrstvou.

4.1.3 Implementácia servisov

Servisná vrstva obsahuje hlavnú logiku celej aplikácie. Servisy sú implementované ako samostatné triedy, ktoré sú do controllerov vkladané pomocou dependency injection. Táto vrstva prijíma dáta od používateľa, overuje ich platnosť a následne ich spracováva. Pri spracovaní komunikuje s databázou cez Entity Framework a pripravuje odpoveď, ktorá je odoslaná späť na frontend. Pri implementácii servisov sa používajú interfacy, ktoré určujú, aké metódy trieda musí mať bez toho, aby prezrádzali, ako sú tieto metódy realizované. Controllery tak vedia, aké funkcie servis ponúka bez toho, aby poznali jeho vnútornú logiku.

4.1.4 Autentifikácia a autorizácia používateľov

Autentifikácia používateľov v aplikácii je založená na JWT tokenoch, čo je moderný a bezpečný spôsob overovania identity používateľa bez potreby udržiavať stav na serveri. Po úspešnom prihlásení backend vygeneruje access token a refresh token. Pri

každej požiadavke klient posiela aj access token. Ak je token platný, používateľ je overený a požiadavka sa spracuje. Ak token nie je platný, server vráti HTTP status kód 401 – Unauthorized. V takom prípade klient využije refresh token a pošle ho na server, aby získal nový access token (a prípadne aj nový refresh token). Ak je refresh token platný, server vráti nové tokeny a používateľ môže pokračovať v práci bez opätovného prihlasovania. Ak je však aj refresh token neplatný alebo expirovaný, klient je odhlásený a používateľ sa musí znovu prihlásiť. Access token má bežne krátku dobu expirácie, v našom prípade 15 minút. Cieľom tohto je minimalizovať riziko zneužitia, keďže ako už vieme, tento token je bezstavový, čo znamená, že nie je uložený nikde na serveri, a preto sa v prípade jeho odcudzenia nedá zrušiť alebo pozmeniť na serveri. Refresh token má naopak dlhšiu platnosť, v našom prípade 7 dní. Máme ho uložený na serveri, konkrétne v databáze, tým pádom sa v prípade odcudzenia dá jednoducho zneplatniť a nemôže byť zneužitý.

Autorizácia zabezpečuje, že používateľ má prístup iba k tým funkcionalitám, na ktoré má oprávnenie. Na ochranu jednotlivých endpointov sú v controlleroch použité autorizačné atribúty, ktoré kontrolujú používateľskú rolu. Tieto používateľské roly boli popísané v časti 3.3 .

4.2 *IMPLEMENTÁCIA FRONTENDU*

4.2.1 Štruktúra frontend projektu

Tak ako aj v backendovej časti začneme vytvorením základnej štruktúry priečinkov pre prehľadnosť celej aplikácie. Aplikácia sa skladá z priečinkov app, components, api, hooks, types, lib.

app/ obsahuje jednotlivé stránky aplikácie a riadi routing. V Next.js je routing založený na konvencii, čo znamená, že URL cesta stránky zodpovedá jej umiestneniu v adresárovej štruktúre projektu

components/ obsahuje znovupoužiteľné UI komponenty, napr. formuláre, tlačidlá, inputy. Vďaka týmto komponentom je možné rýchlo skladať nové stránky bez duplikácie kódu a zachovať jednotný dizajn aplikácie

api/ obsahuje metódy, ktoré komunikujú s backendovým API prostredníctvom Axiosu a zároveň aj z backendu prijímajú dáta

hooks/ obsahuje vlastné react hooky, v našom konkrétnom prípade pre autentifikáciu

types/ obsahuje Typescript typy a rozhrania, ktoré definujú štruktúru dát používaných v aplikácii. Týmto sa zabezpečí typová bezpečnosť a konzistencia dát naprieč celým projektom

lib/ obsahuje pomocné funkcie, využívané naprieč aplikáciou

4.2.2 Routing

V Next.js je routing založený na konvencii, čo znamená, ako bolo spomenuté, že URL adresa stránky priamo zodpovedá jej umiestneniu v adresárovej štruktúre projektu. Routy, ktoré sú určené len pre používateľov s konkrétnymi oprávneniami, sú chránené pomocou overenia prihlásenia a používateľskej roly.

4.2.3 UI a UX

Pri implementácii používateľského rozhrania bol kladený dôraz hlavne na konzistentný dizajn a jednoduchú orientáciu v aplikácii. Na štylovanie som využil Tailwind CSS, vďaka ktorému sme mohli implementovať štýly, layout a responzivitu pomocou jednoduchých class. Pre rýchle vytváranie komponentov som použil knižnicu shadcn/ui, ktorá poskytuje hotové UI komponenty, ako sú tlačidlá, formuláre, sidebar či tabuľky. Tieto komponenty vieme jednoducho prispôbiť našim potrebám, napr. pomocou tailwind a preto sú jednou z najpodstatnejších súčastí našej frontendovej architektúry. Na vizualizáciu a prehľadné zobrazovanie štatistík úspešnosti používateľov sme využili knižnicu Recharts, ktorá umožňuje vytvárať interaktívne grafy a diagramy priamo vo frontendovej časti aplikácie. Recharts nám poskytuje flexibilitu pri vytváraní rôznych typov grafov, ako sú stĺpcové, koláčové alebo čiarové grafy, ktoré sa dynamicky aktualizujú na základe dát získaných z backendu.

4.2.4 Formuláre a validácia

Pri implementácii formulárov v aplikácii sme využili knižnicu TanStack Form, ktorá umožňuje jednoduché spravovanie stavov vstupných polí, odosielanie údajov a kontrolu chýb. Na overovanie správnosti vstupov som použil knižnicu Zod, ktorá umožňuje definovať presné pravidlá, aké hodnoty sú akceptovateľné pre jednotlivé polia, pri chybných vstupoch upozorniť používateľa výpisom na stránku.

4.2.5 Správa dát

Komunikácia s backendom bola riešená v priečinku api. Nachádzajú sa tam funkcie, ktoré pomocou axios posielajú requesty na backend a vracajú dáta, ktoré získali z backendu späť. Pre efektívnu správu dát som využil TanStack Query. Implementovali sme automaticky caching dát, ich obnovu pri zmene alebo po určitom čase (refetching) a invalidácie query, keď sa zmení obsah na serveri, čo znamená, že dáta budú taktiež refetchnuté a používateľ vždy vidí aktuálne údaje. Vďaka tomu sa znižuje počet zbytočných požiadaviek na backend, čo zrýchľuje aplikáciu.

4.2.6 Autentifikácia a autorizácia

Access a refresh token sú na frontende uložené v http only cookies. Access token je na backend odosielaný s každým requestom. V prípade neplatnosti sa klient pokúsi pomocou refresh tokenu získať nové tokeny. V prípade úspechu vie používateľ normálne pokračovať vo využívaní aplikácie, zatiaľ čo v opačnom prípade bude odhlásený a presmerovaný na stránku s loginom.

4.2.7 Typová bezpečnosť

Rovnako ako na backende, aj na frontende sme implementovali typovú bezpečnosť pomocou TypeScriptu. Každý komponent, funkcia či dátová štruktúra má presne definované typy, čo zabraňuje náhodným chybám pri priradení alebo používaní nesprávnych hodnôt.

5 DEPLOY APLIKÁCIE

Deploy aplikácie zahŕňa nasadenie backendovej a frontendovej časti spolu s databázou na server tak, aby bola aplikácia prístupná používateľom cez webový prehliadač. V projekte som sa zameriaval na jednoduché a bezpečné nasadenie tohto systému na web. Zároveň bude pre túto aplikáciu zakúpená aj doména, ktorá zabezpečí jednoduchý prístup k webu.

Záver

Cieľom tohto projektu bolo vytvoriť online testovací modul, ktorý by pomohol maturantom lepšie sa pripraviť na maturitnú skúšku. Počas práce sa mi podarilo navrhnúť a vytvoriť funkčný systém, ktorý umožňuje riešenie testov, zobrazenie správnych odpovedí a sledovanie úspešnosti používateľa. Stanovené ciele práce boli splnené.

Projekt mi umožnil prakticky využiť vedomosti, ktoré som nadobudol počas štúdia, ale najmä tie nadobudnuté samovzdelávaním v oblasti informačných technológií a práce s webovými aplikáciami. Vytvorený systém má reálne využitie a môže pomôcť študentom pri príprave na maturitu alebo písomné testy.

Do budúcnosti by bolo možné projekt rozšíriť o ďalšie predmety, nové typy otázok alebo o aplikáciu pre mobil a podobne. Projekt môže slúžiť ako pomôcka vo výučbe alebo ako základ pre ďalší rozvoj podobných vzdelávacích aplikácií.

Použitá literatura

[HTTPS://LEARN.MICROSOFT.COM/EN-US/ASPNET/CORE/GET-STARTED?VIEW=ASPNETCORE-9.0](https://learn.microsoft.com/en-us/aspnet/core/get-started?view=aspnetcore-9.0)

[HTTPS://LEARN.MICROSOFT.COM/EN-US/EF/CORE/](https://learn.microsoft.com/en-us/ef/core/)

[HTTPS://WWW.W3SCHOOLS.COM/POSTGRESQL/INDEX.PHP](https://www.w3schools.com/postgresql/index.php)

[HTTPS://NEXTJS.ORG/DOCS](https://nextjs.org/docs)

[HTTPS://TANSTACK.COM/FORM/V1](https://tanstack.com/form/v1)

[HTTPS://ZOD.DEV/](https://zod.dev/)

[HTTPS://TANSTACK.COM/QUERY/V5+](https://tanstack.com/query/v5+)

[HTTPS://AXIOS-HTTP.COM/DOCS/INTRO](https://axios-http.com/docs/intro)

[HTTPS://UI.SHADCN.COM/](https://ui.shadcn.com/)

[HTTPS://TAILWINDCSS.COM/](https://tailwindcss.com/)

[HTTPS://LUCIDE.DEV/GUIDE/PACKAGES/LUCIDE-REACT](https://lucide.dev/guide/packages/lucide-react)

[HTTPS://RECHARTS.GITHUB.IO/EN-US/EXAMPLES/](https://recharts.github.io/en-US/examples/)

[HTTPS://WWW.NPMJS.COM/PACKAGE/NEXT-THEMES](https://www.npmjs.com/package/next-themes)

[HTTPS://WWW.YOUTUBE.COM/@MILANJOVANOVICTECH](https://www.youtube.com/@MilanJovanovicTech)

[HTTPS://WWW.YOUTUBE.COM/WATCH?V=GTGBH9ALW5G&LIST=PL3InzUXLS85Nd9BMNL1XrWE d8TPJLSQDQ](https://www.youtube.com/watch?v=GTGBH9ALW5G&list=PL3InzUXLS85Nd9BMNL1XrWE d8TPJLSQDQ)

Príloha A

Link na webovú stránku: otestujsa.site

Link na zip súbor stránky:

<https://www.mediafire.com/file/7qtguhi2b5lt0ou/Matúš+Čiernik.zip/file>